

End-to-End Cloud-Based Medical Chat Assistant

1. System Architecture

1.1 System Architecture Diagram

Figure 1 presents the complete system architecture for the cloud-based medical chat assistant. The diagram illustrates all infrastructure components and their interconnections, including the custom VPC (25cdkg-vpc, CIDR 10.0.0.0/16), two subnets, dedicated security groups for both the EMR cluster and EC2 instance, the Amazon EMR cluster used for data preprocessing, the EC2 instance (g4dn.xlarge) hosting the LLM runner (Ollama) and the web-based chat interface (OpenWebUI), and the S3 bucket serving as the central data lake. All traffic flows are annotated with their respective ports and protocols.

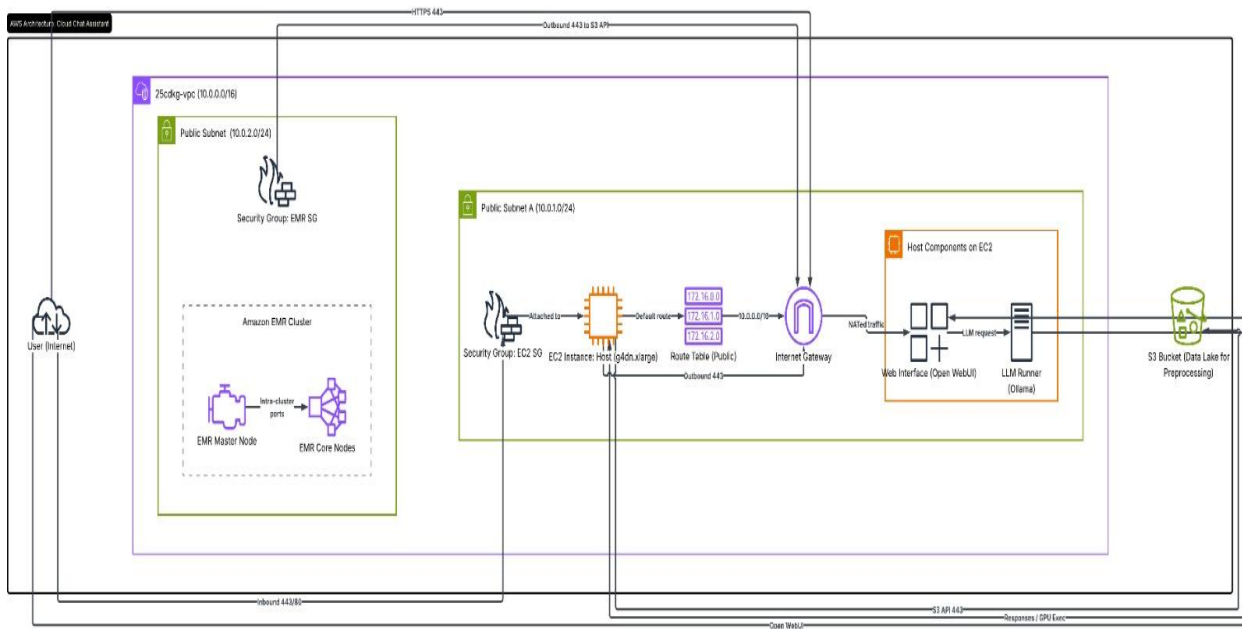


Figure 1. End-to-end system architecture for the cloud-based medical chat assistant, showing all AWS components, network configuration, and data flow paths from preprocessing through model serving to user interaction.

1.2 Data Flow Overview

The system follows a five-stage pipeline that transforms raw medical data into an interactive chat experience:

- 1. Data Ingestion & Storage:** The raw MedQuad dataset is uploaded to an Amazon S3 bucket (25cdkg-medical-qa), which serves as the central data lake for the entire pipeline.
- 2. Distributed Preprocessing:** An Amazon EMR cluster (25cdkg-emr-preprocessing) running Apache Spark reads the raw data from S3 and executes a preprocessing pipeline that includes text normalization, deduplication, outlier removal, and ChatML prompt formatting. The cleaned and split datasets (train/validation/test) are written back to S3.
- 3. Model Fine-Tuning:** The preprocessed training data is used to fine-tune a 1.5B-parameter LLM (Qwen 2.5 Instruct) using LoRA (Low-Rank Adaptation) via the Unsloth library on Google Colab. The resulting model is exported to GGUF format for efficient deployment.

4. **Model Serving:** The GGUF model is deployed to an EC2 instance (g4dn.xlarge) within the custom VPC. Ollama acts as the inference engine, loading the model into GPU memory and exposing an HTTP API on port 11434 for programmatic access.
5. **User Interaction:** OpenWebUI, deployed as a Docker container on the same EC2 instance, connects to the Ollama API and provides a browser-based chat interface accessible on port 8080. Users interact with the medical assistant through this interface, with all traffic routed through the Internet Gateway attached to the public subnet.

All components operate within a custom VPC with carefully scoped security group rules, ensuring that only necessary ports are exposed and that internal communication between services remains isolated from the public internet.

2. VPC & Networking

2.1 VPC Configuration

A dedicated Virtual Private Cloud was provisioned from scratch for this project. The default AWS VPC was deliberately not used, as required by the project deliverable, to ensure full control over network isolation, subnet placement, and security group configuration.

Attribute	Value
VPC Name	25cdkg-vpc
VPC ID	vpc-09e955535b25be273
CIDR Block	10.0.0.0/16
Region	Us-east-1(N.Virginia)
DNS Resolution	Enabled
Tenancy	Default
Default VPC	NO

Design Justification: A /16 CIDR block was chosen to provide up to 65,536 private IP addresses, which is more than sufficient for the project's current components (EMR cluster, EC2 instance, and supporting services) while leaving ample room for future expansion. A custom VPC was selected over the default VPC because it provides full control over subnet segmentation, routing policies, and security group scoping — all of which are essential for isolating the data preprocessing environment (EMR) from the public-facing model serving layer (EC2).

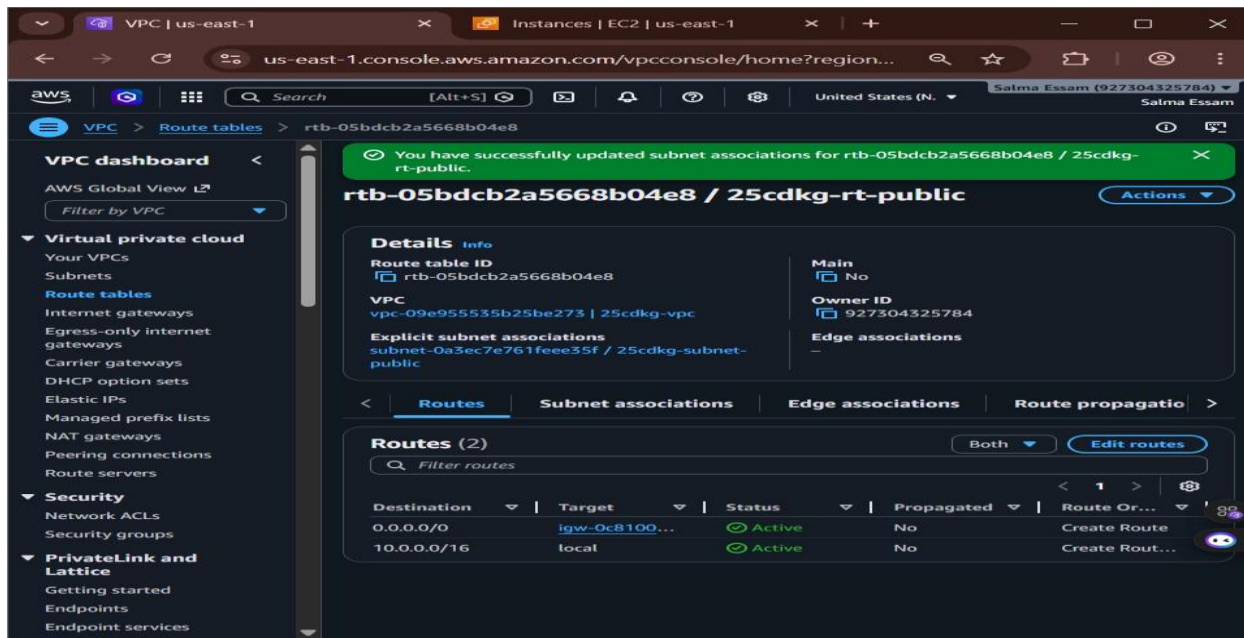


Figure 2. Successful creation of the custom VPC (25cdkg-vpc) on AWS. The screenshot confirms the VPC ID (vpc-09e95535b25be273), CIDR block (10.0.0.0/16), "Available" state, DNS Resolution enabled, and that this is explicitly not the default VPC.

2.2 Subnet Design

A dual-subnet architecture was implemented to enforce network-level separation between public-facing services and internal data processing workloads:

Subnet	Name	CIDR	Availability	Purpose	Internet Access
Public	25cdkg-subnet-public	10.0.3.0/24	Available	EC2 instance hosting Ollama and OpenWebUI	Yes (via IGW)
Private	25cdkg-subnet-private	10.0.2.0/24	Available	EMR cluster for Spark preprocessing	No direct access

Design Justification: The EC2 instance hosting Ollama and the OpenWebUI chat interface must be publicly accessible so that end users can reach the medical assistant through their browsers; therefore, it is placed in the public subnet with an associated Internet Gateway route. The EMR cluster, which processes sensitive medical data during the preprocessing phase, is placed in the private subnet to ensure that its nodes are never directly exposed to the public internet. This separation adheres to the principle of least exposure and minimises the infrastructure's attack surface. Each subnet uses a /24 CIDR block, providing up to 251 usable IP addresses per subnet — sufficient for the project's needs while maintaining clear address space boundaries.

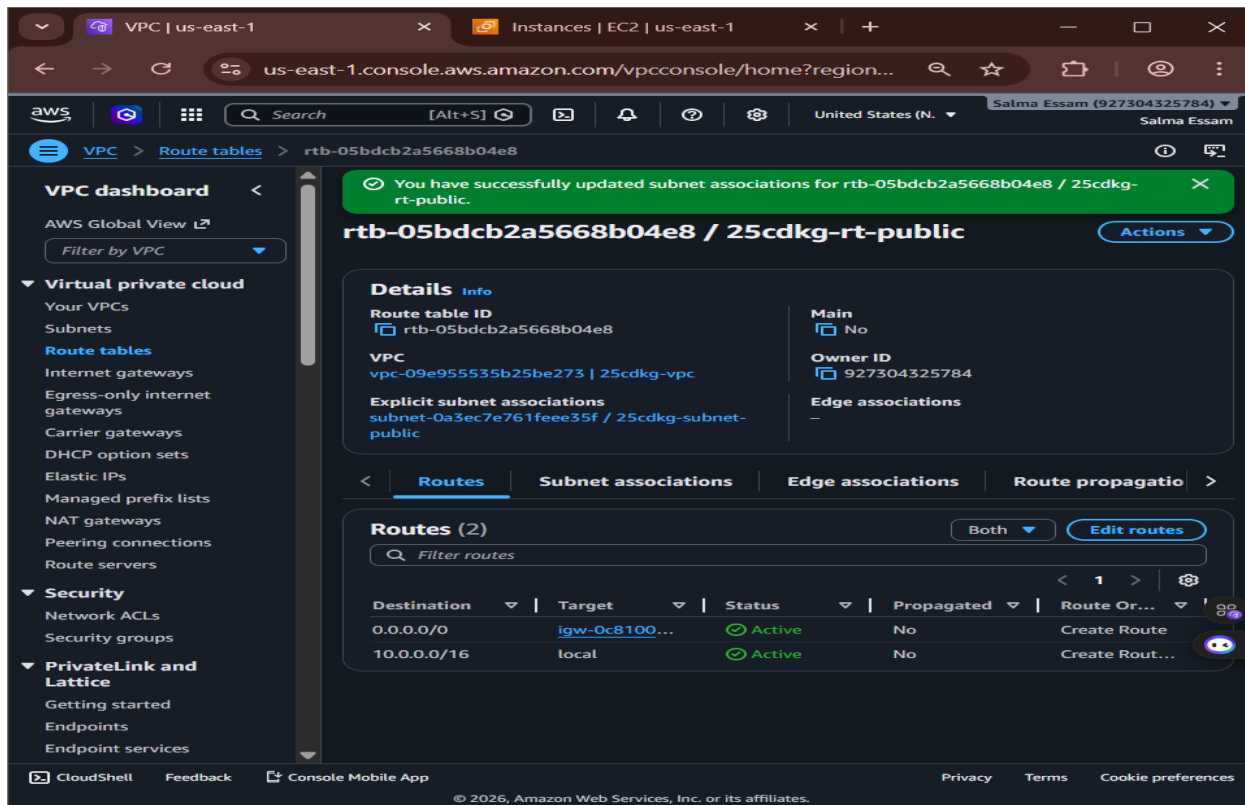


Figure 3. Successful creation of the public subnet (25cdkg-subnet-public, ID: subnet-0a3ec7e761fee35f) within the custom VPC, in "Available" state.

2.3 Internet Gateway & Route Tables

An Internet Gateway (IGW) was created and attached to the VPC to enable bidirectional internet communication for resources residing in the public subnet.

Component	Name	ID	Status
Internet Gateway	25cdkg-igw	igw-0c81005f68ec2cd76	Attached to 25cdkg-vpc
Route Table	25cdkg-rt-public	rtb-05bdc2a5668b04e8	Associated with 25cdkg-subnet-public

Route Table Configuration

Destination	Target	Status	Purpose
0.0.0.0/0	igw-0c81005f68ec2cd76	Active	Routes all internet-bound traffic through the Internet Gateway
10.0.0.0/16	Local	Active	Keeps all intra-VPC traffic within the local network

Design Justification: The default route (0.0.0.0/0 → IGW) serves two critical functions: (1) it allows end users to access the OpenWebUI chat interface from external browsers, and (2) it enables the EC2 instance to download required software packages — including Ollama, Docker, and Python libraries — during initial provisioning and setup. The local route (10.0.0.0/16 → local) ensures that all communication between the EC2 instance and EMR cluster nodes remains within the VPC and never traverses the public internet, preserving both performance and security. The route table is explicitly associated with the public subnet only; the private subnet retains its default route table without an IGW route, ensuring that EMR nodes cannot initiate outbound internet connections.

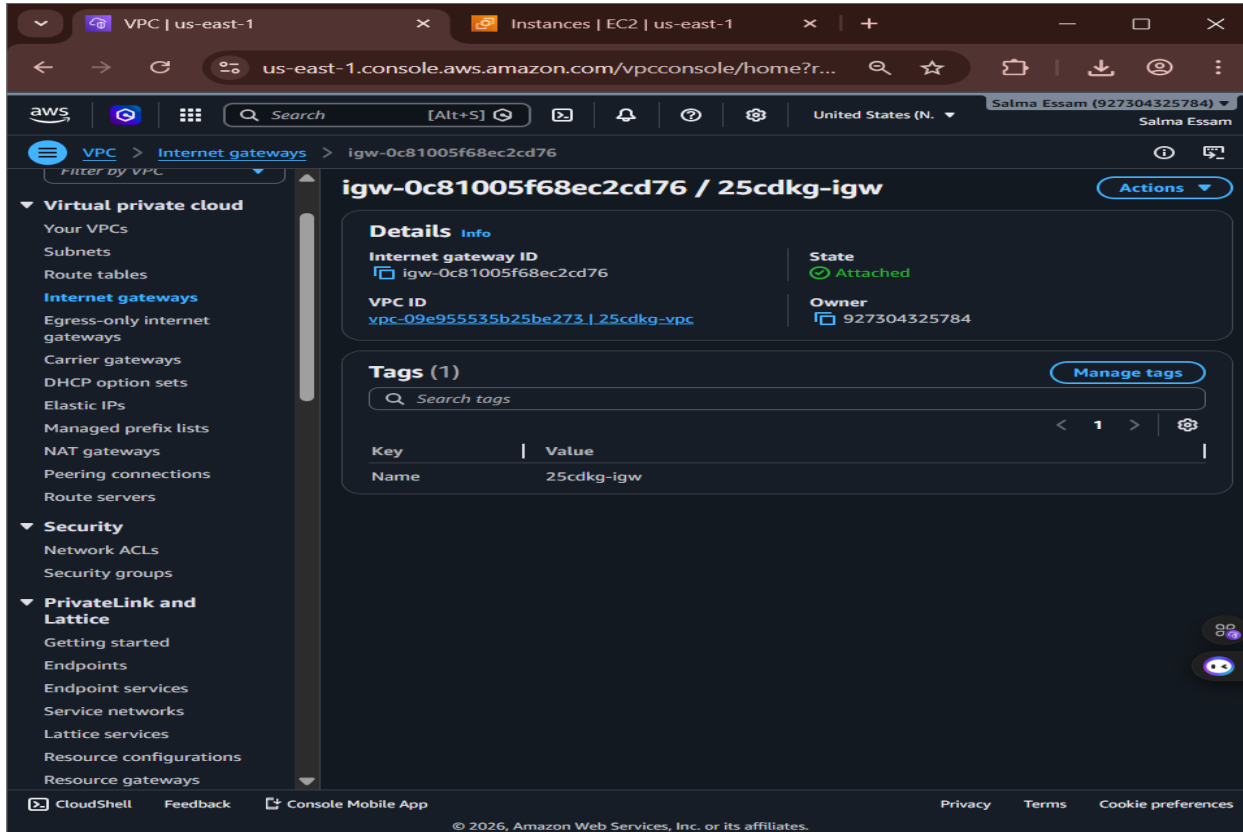


Figure 4. Internet Gateway (25cdkg-igw, ID: igw-0c81005f68ec2cd76) in "Attached" state, connected to 25cdkg-vpc.

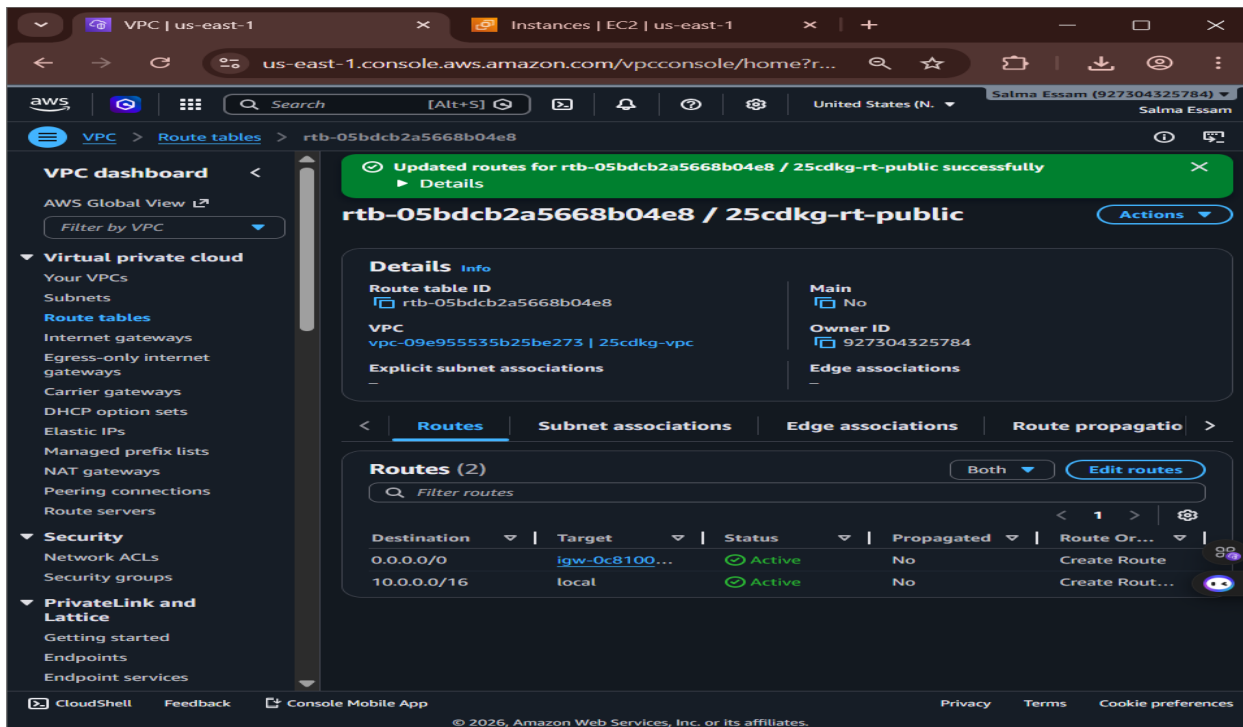


Figure 5. Public route table (25cdkg-rt-public) configuration showing two route entries: the default route (0.0.0.0/0) directed to the Internet Gateway and the local route (10.0.0.0/16) for intra-VPC traffic. The route table is explicitly associated with the public subnet (25cdkg-subnet-public).

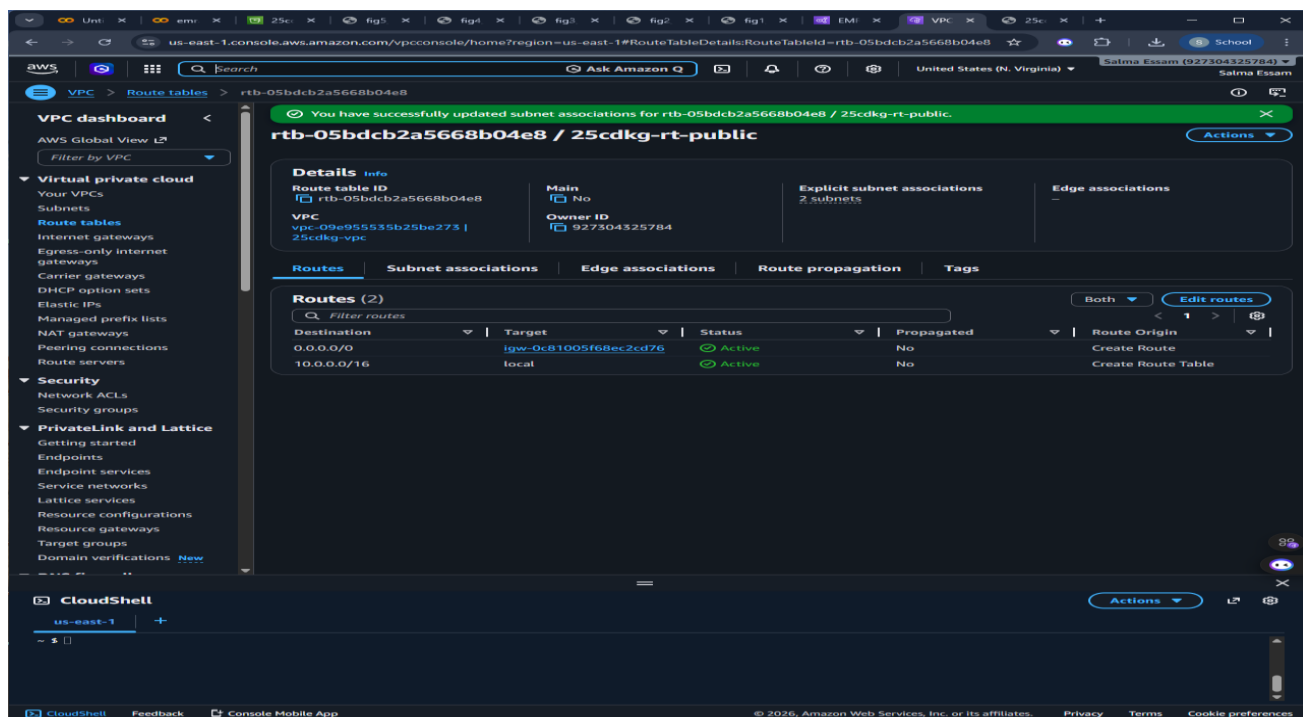


Figure 6. Confirmation of the explicit subnet association between the public route table (25cdkg-rt-public) and the public subnet (25cdkg-subnet-public).

2.4 Security Groups

Two dedicated security groups were created following the principle of least privilege, each scoped to a specific infrastructure component. Every inbound rule is documented with its port, source, purpose, and the reasoning behind its inclusion.

Security Group 1: EC2 Instance — 25cdkg-sg-ec2

Attribute	Value
Security Group Name	25cdkg-sg-ec2
Security Group ID	sg-0a6601cd277b5091c
Description	Security Group for EC2 instance — Ollama and OpenWebUI
VPC	vpc-09e955535b25be273 (25cdkg-vpc)
Inbound Rules Count	3
Outbound Rules Count	4

Inbound Rules

port	Protocol	Source	Purpose	Justification
22	TCP	My IP only	SSH Access	Restricts administrative terminal access exclusively to the operator's IP address, preventing unauthorized remote login attempts.
3000	TCP	0.0.0.0/0	OpenWeb UI Interface	Opens the web-based chat interface to all users so that the medical assistant can be accessed from any browser. Port 3000 is the default port for the OpenWebUI Docker container.
11434	TCP	10.0.0.0/16 (VPC)	Ollama API	Enables internal communication between the OpenWebUI container and the Ollama inference engine. This port is scoped to VPC-internal traffic only and is not exposed to the public internet, ensuring that the model API cannot be called directly by external clients.

Outbound Rules

Type	Protocol	Destination	Purpose
Custom TCP	TCP	Various	Application-specific outbound traffic
All Traffic	All	0.0.0.0/0	General outbound (package installation, S3 access)
SSH	TCP	Various	Outbound SSH for management
Custom TCP	TCP	Various	Additional service communication

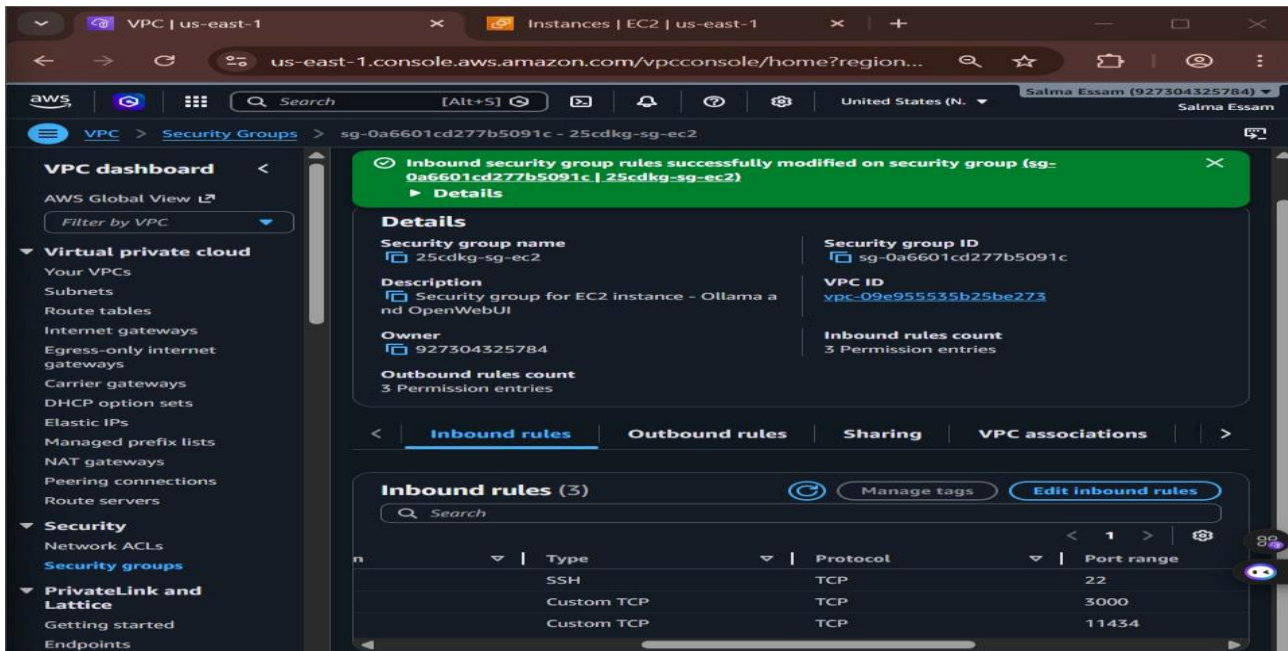


Figure 7. Inbound security group rules for the EC2 instance (25cdkg-sg-ec2) permitting SSH (port 22), OpenWebUI (port 3000), and Ollama API (port 11434).

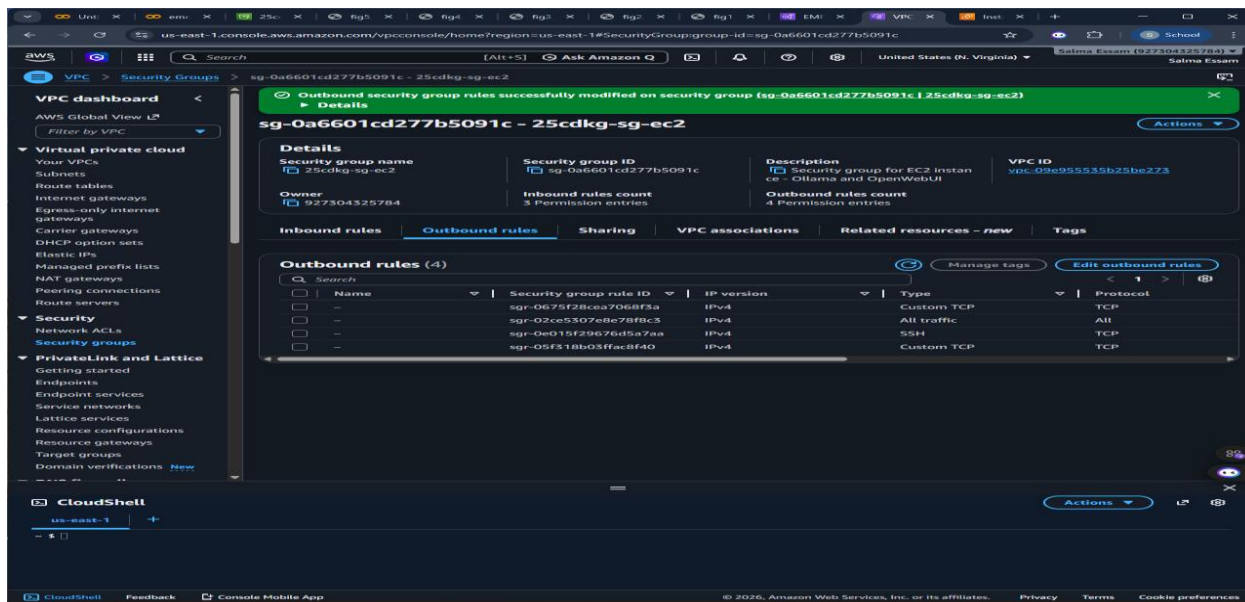


Figure 8. Outbound security group rules for the EC2 instance (25cdkg-sg-ec2) showing four outbound permission entries for application traffic, general internet access, and service communication.

Security Group 2: EMR Cluster — 25cdkg-sg-emr

Security Group Name	25cdkg-sg-emr
Security Group ID	sg-09d12a1eed84ecc02
Description	Security Group for EMR cluster — allows SSH and internal VPC traffic
VPC	vpc-09e955535b25be273 (25cdkg-vpc)
Inbound Rules Count	2
Outbound Rules Count	1

Inbound Rules

Port	Protocol	Source	Purpose	Justification
22	TCP	My IP only	SSH Access	Allows administrative access to the EMR primary node for monitoring Spark job execution, reviewing logs, and debugging preprocessing pipeline failures.
All TCP	TCP	Self (same SG)	Intra-cluster Communication	EMR nodes (Primary, Core, Task) require unrestricted TCP communication among themselves for HDFS block replication, YARN resource negotiation, Spark shuffle data exchange, and inter-node heartbeat monitoring. Restricting this traffic would cause cluster instability and job failures.

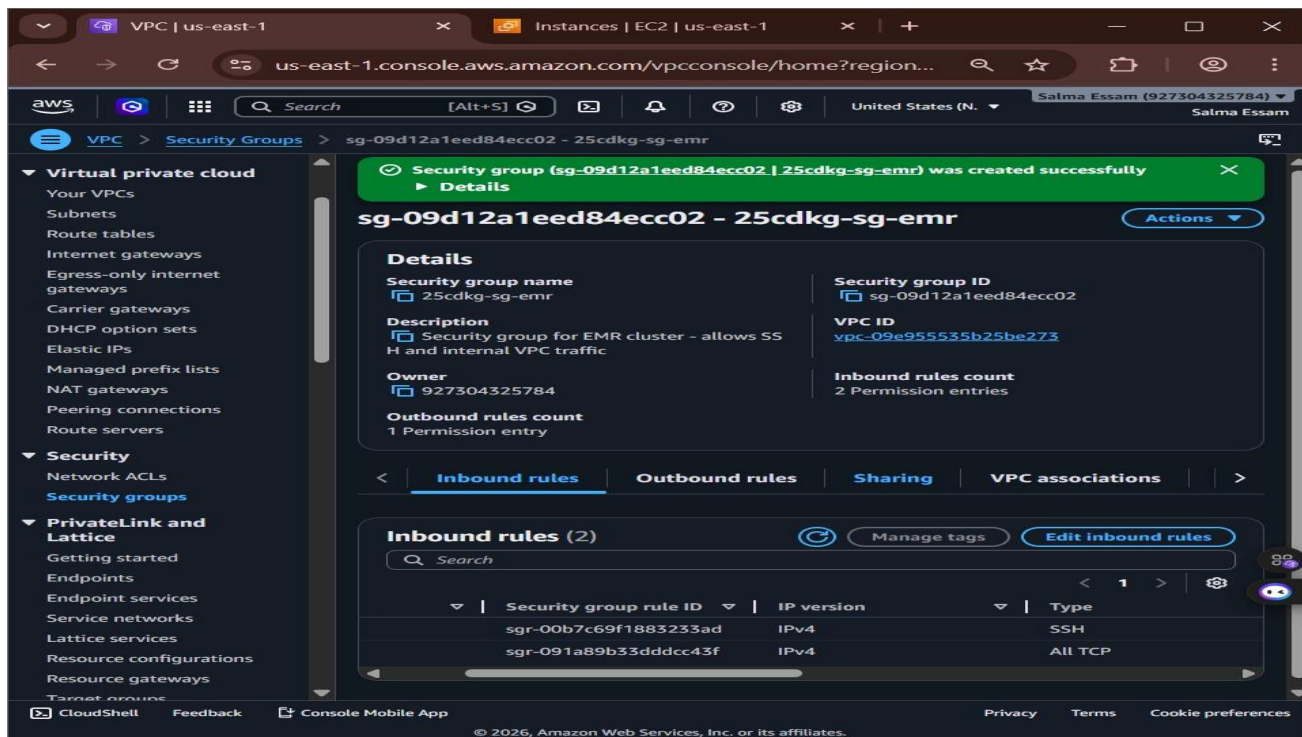


Figure 9. Inbound security group rules for the EMR cluster (25cdkg-sg-emr) permitting SSH access and all intra-cluster TCP communication required for distributed Spark processing.

3. Model & Dataset Selection

3.1 Model Selection

The model selected for this project is a quantised variant of the Qwen 2.5 Instruct series, sourced from the Unsloth repository on Hugging Face Hub.

Model Name	unsloth/qwen2.5-1.5b-instruct-bnb-4bit
Parameters	1.5 Billion
Source	Unsloth (Hugging Face Hub)
Link	huggingface.co/unsloth/qwen2.5-1.5b-instruct-bnb-4bit
License	Apache 2.0
Quantization	4-bit (BitsAndBytes) — loaded with load_in_4bit=True
Max Sequence Length	2,048 tokens

Hardware Fit: At 1.5 billion parameters with 4-bit quantisation, the model's memory footprint is approximately 0.9 GB, which fits comfortably within the 16 GB VRAM budget of a free Google Colab T4 GPU. This is well within the project guideline of selecting an open-source model under 10 billion parameters to avoid hardware limitations during fine-tuning. The 4-bit quantisation via BitsAndBytes further reduces VRAM consumption, eliminating out-of-memory errors and enabling rapid iteration during training.

Domain Fit: The Qwen 2.5 Instruct series natively uses a ChatML-style conversation format with `<|im_start|>` and `<|im_end|>` delimiter tokens to delineate system, user, and assistant turns. This architecture aligns directly with the prompt template applied during the PySpark preprocessing stage (Section 4), ensuring zero format mismatch between training data and model expectations. This architectural compatibility makes the model particularly well-suited to the structured medical Q&A format of the MedQuad dataset.

3.2 Dataset Selection

Dataset Name	MedQuad (Medical Question & Answer Dataset)
Source	KaggleHub — keivalya/MedQuad-MedicalQnADataset
Link	https://www.kaggle.com/datasets/thedevastator/comprehensive-medical-q-a-dataset?resource=download
License	CC0 1.0 Universal (Public Domain)
Total Samples(RAW)	38,088 rows (verified via df.count() in PySpark)
After Preprocessing	17,364 clean samples (after null removal, deduplication, and outlier filtering)
Question Categories	31 distinct qtype values (e.g., Treatment, Diagnosis, Susceptibility, Symptoms)
Columns	qtype, Question, Answer

Domain Fit: The dataset contains over 38,000 expert-validated medical question-answer pairs spanning 31 distinct clinical categories, including treatments, chronic disease management, diagnostic procedures, and symptom descriptions. The breadth and quality of this data make it an ideal candidate for fine-tuning a domain-specific medical chat assistant. The CC0 1.0 license permits unrestricted use for academic and research purposes.

3.3 Dataset Characteristics

The dataset consists of three columns:

- **qtype**: The medical category of the question (e.g., Treatment, Diagnosis, Susceptibility).
- **Question**: The patient inquiry, typically ranging from short factual questions to multi-sentence clinical queries.
- **Answer**: The expert-validated medical response, sourced from healthcare professionals including doctors, nurses, and pharmacists.

Verbatim Sample from the Raw Dataset

The following is an unmodified sample taken directly from the dataset to illustrate its structure and content:

qtype	susceptibility
Question	Who is at risk for Lymphocytic Choriomeningitis (LCM)?
Answer	LCMV infections can occur after exposure to fresh urine, droppings, saliva, or nesting materials from infected rodents. Transmission may also occur when these materials are directly introduced into broken skin, the nose, the eyes, or the mouth, or presumably, via the bite of an infected rodent. Person-to-person transmission has not been reported, with the exception of vertical transmission from infected mother to fetus, and rarely, through organ transplantation.

Summary Statistics

The following statistics were computed after dropping null values (23,831 remaining rows), prior to outlier filtering: After applying outlier filters (removing answers exceeding 500 words and questions shorter than 5 words) and deduplicating identical question-answer pairs, the final cleaned dataset contains **17,364 samples**.

Detailed exploratory data analysis — including token length distributions, label balance across question types, and question-answer length correlations — is presented in Section 4.

```
After dropping nulls: 23831 rows
Summary Statistics for Token/Word Counts:
+-----+-----+-----+
|summary|          q_len|          a_len|
+-----+-----+-----+
|  count|          23831|          23831|
|   mean|10.258780579916914|105.11703243674206|
|  stddev|13.481457245933242| 126.3072343140169|
|   min|              1|              1|
|   max|             214|            1960|
+-----+-----+-----+
```

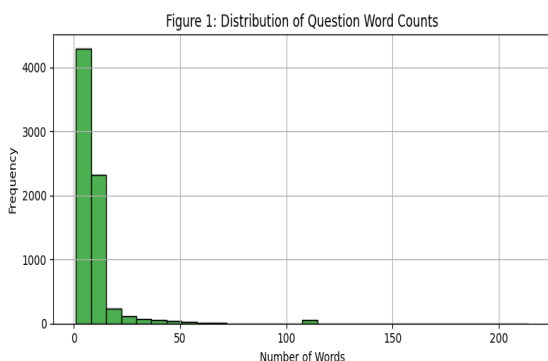


Figure 10. Distribution of question word counts in the raw dataset. The majority of questions are concise (under 20 words), with a small number of outliers exceeding 100 words. This distribution informed the decision to filter questions shorter than 5 words during preprocessing.

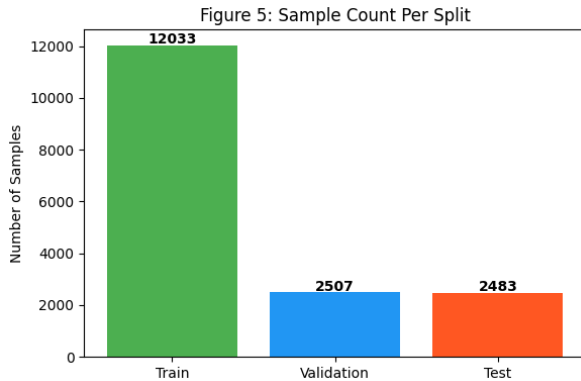


Figure 11. Distribution of samples across the train/validation/test splits after preprocessing: Train (12,033), Validation (2,507), Test (2,483), confirming the intended 70/15/15 ratio.

3.4 Data Splitting Strategy

To ensure robust model evaluation and prevent data leakage, the preprocessed dataset was partitioned into three non-overlapping splits using a fixed random seed:

Data Leakage Prevention: Three safeguards were implemented to prevent information leakage between splits:

Training Set	70%	Used for LoRA fine-tuning via Unsloth/SFTTrainer
Validation Set	15%	Used for monitoring training progress and hyperparameter selection
Test Set	15%	Held out entirely for final evaluation (ROUGE/BLEU metrics)

- 1. Deduplication before splitting:** All duplicate question-answer pairs were removed using `dropDuplicates(["Question", "Answer"])` in the PySpark pipeline *prior* to the split operation, ensuring that no identical samples appear across different partitions.
- 2. Fixed random seed:** The `randomSplit([0.7, 0.15, 0.15], seed=42)` function was used with a deterministic seed, guaranteeing reproducible and consistent partitioning across runs.
- 3. No test data exposure during training:** The test set was reserved exclusively for post-training evaluation; it was never used during training or hyperparameter tuning. Validation loss was monitored using only the validation split.

4. Data Preprocessing with Apache Spark on EMR

4.1 EMR Cluster Configuration

To perform distributed data preprocessing at scale, an Amazon EMR cluster was provisioned within the project's custom VPC. Apache Spark was selected as the processing engine due to its ability to handle large-scale text transformations efficiently across multiple nodes.

Cluster Name	25cdkg-emr-cluster
Cluster ID	j-2FXYISNTHZ76L
EMR Version	emr-7.0.0
Application	Spark 3.5.0, Hadoop
Capacity	1 Primary, 1 Core, 1 Task
Instance Type	m5.xlarge (all nodes)

Region	us-east-1
OS	Amazon Linux 2023.11
Log Destination	s3://25cdkg-medical-qa/25cdkg-logs
Auto-Termination	Enabled (1 hour idle)
Termination Protection	Off
Security Group	25cdkg-sg-emr
Status	Terminated (User request)

Design Justification: The **m5.xlarge** instance type (4 vCPUs, 16 GB RAM per node) provides sufficient memory for Spark's in-memory processing of the 38,088-row MedQuad dataset. A three-node cluster (1 Primary + 1 Core + 1 Task) balances processing speed with cost efficiency. Auto-termination after 1 hour of idle time was enabled to prevent unnecessary charges on the shared AWS account.

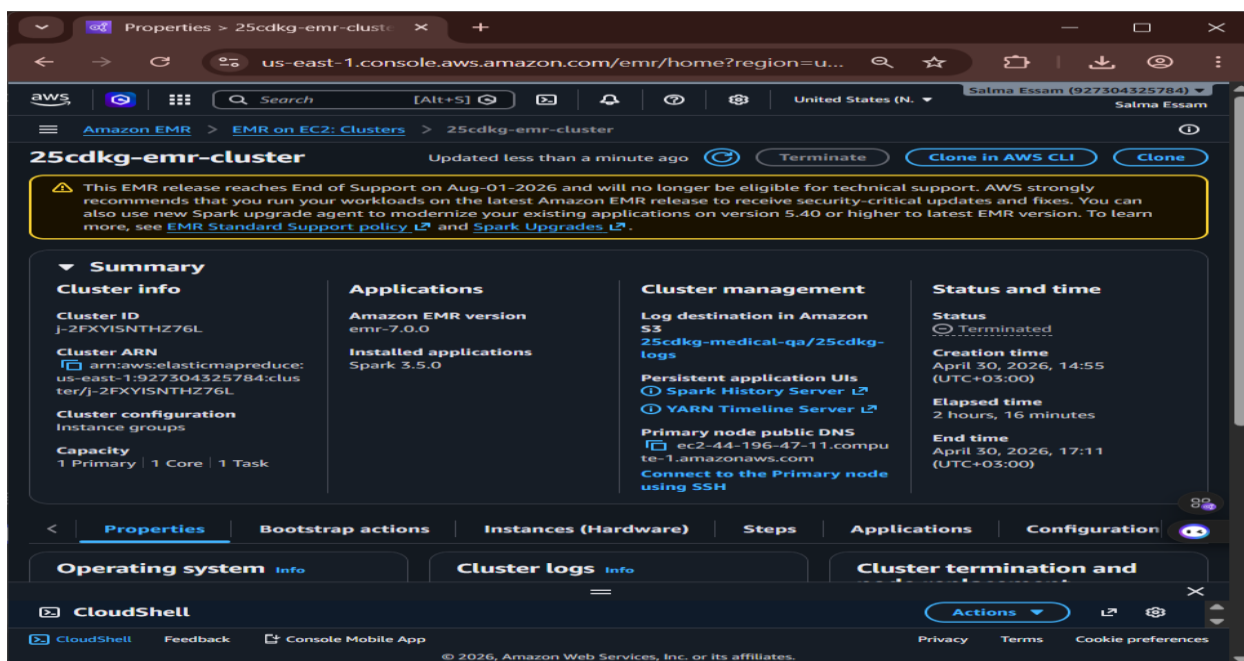


Figure 12. Amazon EMR cluster summary (25cdkg-emr-cluster) showing the cluster configuration, Spark 3.5.0 application, and confirmed "Terminated" status after preprocessing completion.

4.2 PySpark Preprocessing Pipeline

The preprocessing pipeline was implemented as a PySpark script committed to the GitHub repository and submitted as an EMR Step. The pipeline transforms raw medical Q&A data into a clean, formatted dataset ready for LLM fine-tuning. Each step is described below with its rationale.

Step 1 : Data Loading

The raw train.csv file was read from the S3 landing zone using Spark's CSV reader with automatic header detection and schema inference: The loaded DataFrame contained 38,088 rows and 3 columns: qtype, Question, and Answer.

```
Spark session created
Data loaded – 38088 rows, 3 columns
Columns: ['qtype', 'Question', 'Answer']
```

Step 2 : Text Normalization

All text columns were converted to lowercase and stripped of leading/trailing whitespace to produce a consistent text surface for downstream processing. This prevents duplicate entries caused solely by case differences (e.g., "Diabetes" vs. "diabetes")

Step 3 :Null Removal

Rows with missing Question or Answer fields were dropped to prevent errors during prompt formatting and model training

Step 4 : Word Count Derivation

Two derived columns — `q_len` (question word count) and `a_len` (answer word count) — were computed by splitting on whitespace. These columns serve as the basis for both outlier detection and exploratory data analysis

```
After dropping nulls: 23831 rows
Summary Statistics for Token/Word Counts:
+-----+-----+-----+
|summary|          q_len|          a_len|
+-----+-----+-----+
|  count|          23831|          23831|
|   mean|10.258780579916914|105.11703243674206|
| stddev|13.481457245933242| 126.3072343140169|
|   min|              1|              1|
|   max|             214|             1960|
+-----+-----+-----+
```

Step 5 :Outlier Filtering

```
=== Answer Length Statistics ===
+-----+-----+
|summary|  answer_words|
+-----+-----+
|  count|          23831|
|   mean|105.11703243674206|
| stddev| 126.3072343140169|
|   min|              1|
|   max|             1960|
+-----+-----+

Rows > 500 words: 320 (1.3%)
Rows > 250 words: 2991 (12.6%)

→ We choose 500 as the upper limit to keep most data
```

Three filtering criteria were applied to remove samples that would negatively impact training quality if :

- Questions shorter than 5 words are too vague to form meaningful medical queries
- Answers shorter than 10 words lack sufficient medical detail for training
- Answers exceeding 500 words would surpass the model's 2,048-token context window after ChatML formatting, causing truncation during training

Step 6 :Deduplication & Whitespace Normalization

Duplicate question-answer pairs were removed to prevent the model from memorizing repeated examples. Additionally, multiple consecutive whitespace characters were collapsed into single spaces to normalize formatting artifacts from the source data

Step 7 :qtype Normalization

The **qtype** field in the raw data contained colon-delimited composite labels (e.g., "**category: subtype**"). The suffix after the colon was extracted and trimmed to produce a clean **qtype_clean** column used for EDA grouping and label analysis

Step 8 :ChatML Prompt Formatting

Each cleaned row was formatted into the ChatML instruction template expected by Qwen 2.5 Instruct. This ensures that every training sample matches the exact prompt structure the model will encounter during inference

Step 9 : Character-Level Safety Filter

A final safety filter was applied at the character level to complement the word-level filtering performed in Step 5. This catches edge cases where short words might pass the word count threshold but still contain insufficient content

After all preprocessing steps, the final dataset contained 17,364 clean samples.

Step 10 : Data Splitting & Export to S3

The fully preprocessed DataFrame was split into three non-overlapping partitions using a fixed random seed for reproducibility. Each split was exported to S3 as a single JSON file using `coalesce(1)` to produce one partition file per split, simplifying downstream data loading

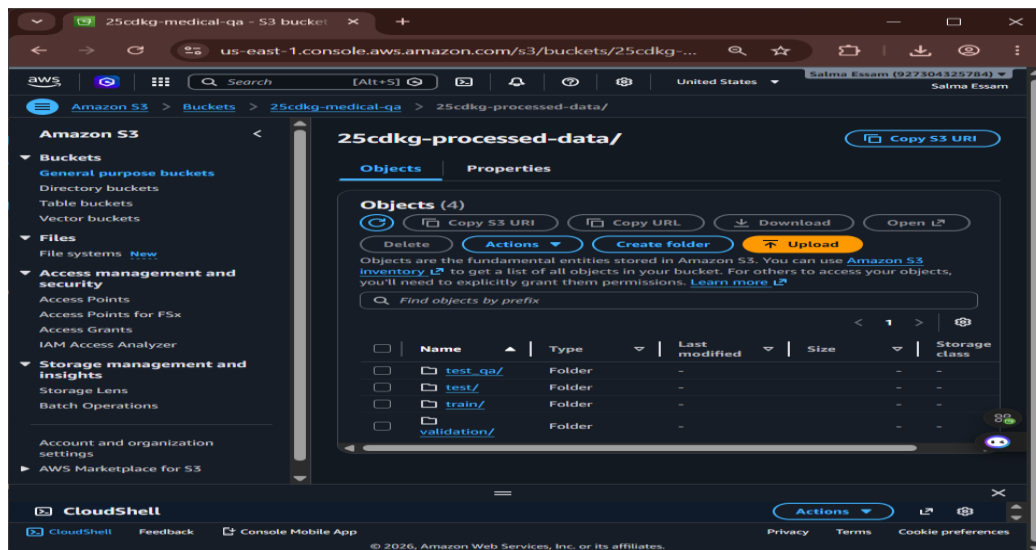


Figure 13. AWS S3 bucket structure showing the preprocessed output files organized into **train/**, **val/**, and **test/** folders within the **25cdkg-processed-data/** prefix, confirming successful export from the EMR Spark pipeline.

4.3 EMR Steps Execution

The preprocessing script was submitted as an EMR Step. After several iterations to resolve dependency and path issues, the final step completed successfully:

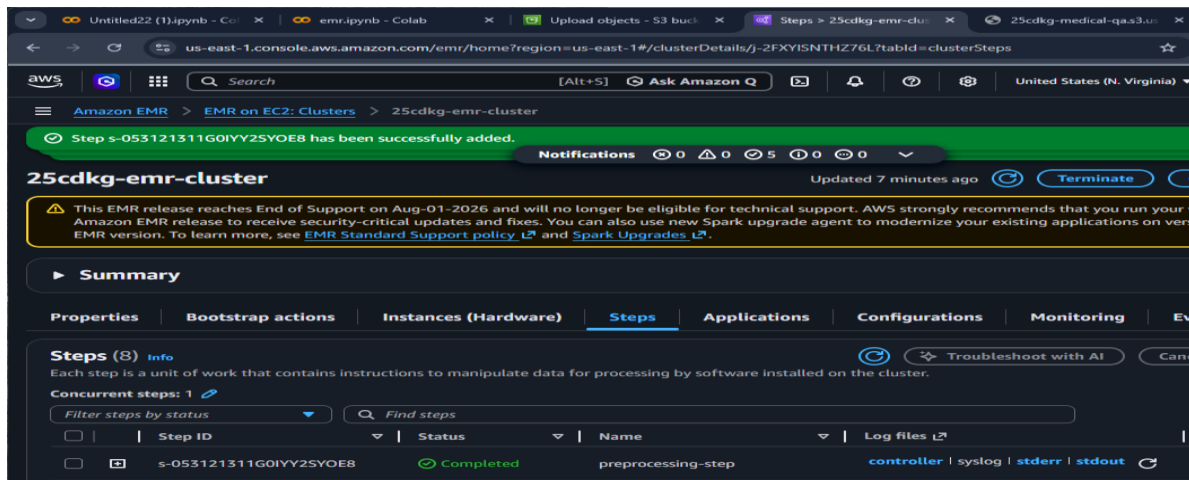


Figure 14. EMR Steps tab showing the final successful execution of the preprocessing step (**preprocessing-step**, Status: Completed), following iterative debugging of earlier failed attempts.

4.4 Exploratory Data Analysis (EDA)

Five EDA figures were generated during preprocessing to characterize the dataset and validate the cleaning decisions. Each figure is presented below with a descriptive caption.

Figure 15. Histogram showing the distribution of question word counts across the dataset. The vast majority of questions are concise, containing fewer than 20 words. A small number of outliers exceed 100 words. This right-skewed distribution confirmed the decision to filter questions shorter than 5 words, as these are unlikely to represent meaningful medical queries.

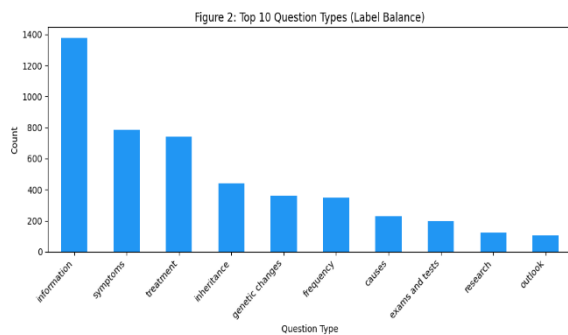
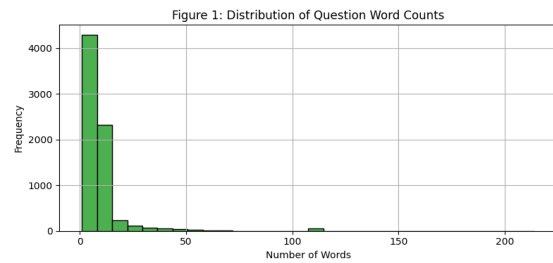


Figure 16. Bar chart representing the top 10 question types (**qtype**) after normalization. The most frequent categories are "information" (~1,400 samples), "symptoms" (~790), and "treatment" (~740). The moderate class imbalance is expected in real-world medical data and does not require explicit balancing for this project's scope.

Figure 17. Scatter plot illustrating the relationship between question length and answer length. Shorter questions (under 25 words) exhibit the widest variance in answer lengths, ranging from a few words to over 1,200 words. This visualization confirmed the need for the 500-word upper bound on answer length to prevent context window overflow during training.

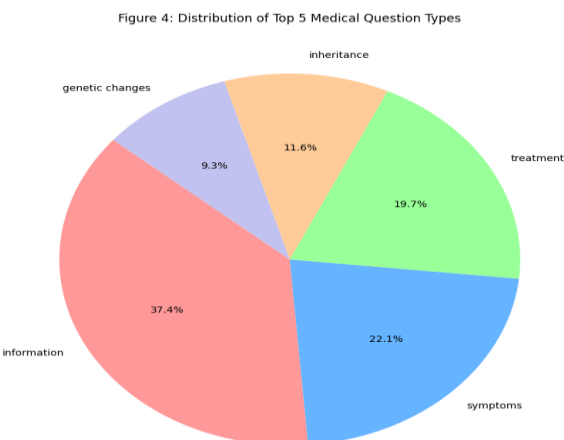
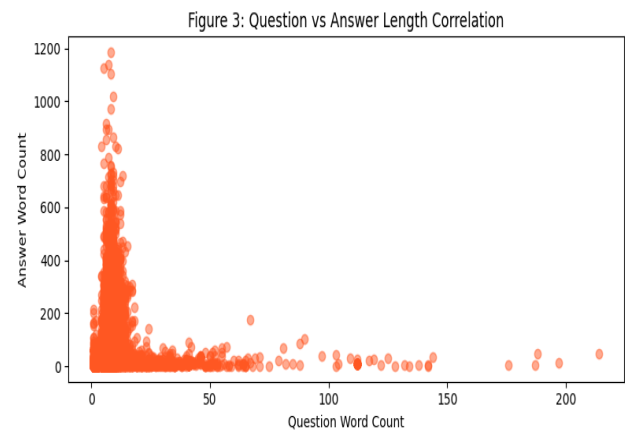
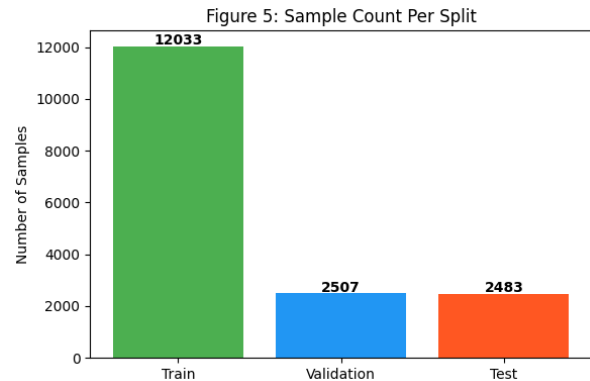


Figure 18. Pie chart showing the proportional distribution of the top 5 medical question categories. "Information" dominates at 37.4%, followed by "symptoms" (22.1%), "treatment" (19.7%), "inheritance" (11.6%), and "genetic changes" (9.3%). This distribution reflects the dataset's emphasis on general medical knowledge and symptom-related queries.

Figure 19. Bar chart confirming the final sample distribution across the three data splits: Train (12,033 samples, 70%), Validation (2,507 samples, 15%), and Test (2,483 samples, 15%). The split ratios align with the intended 70/15/15 strategy described in Section 3.



4.5 EMR Teardown Confirmation

As required by the project deliverable, the EMR cluster was terminated immediately after the preprocessing pipeline completed. The following screenshot confirms the cluster's "Terminated" status:

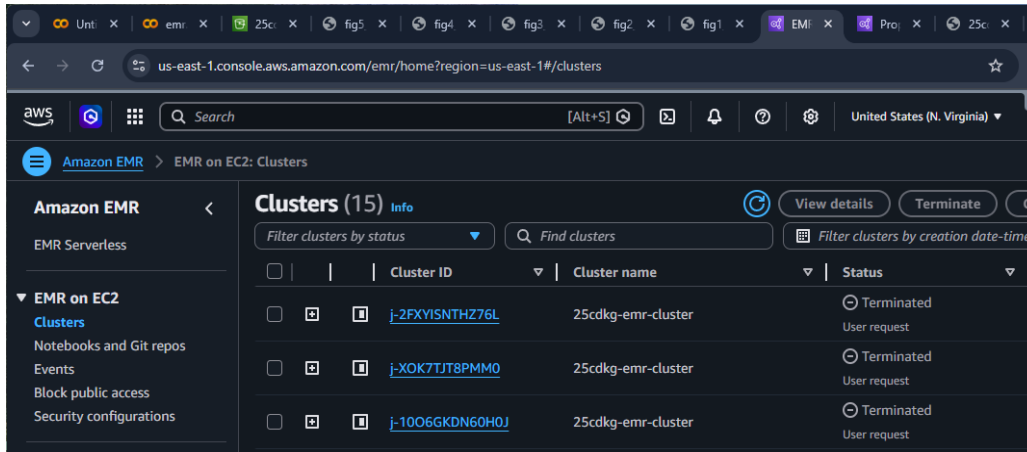


Figure 20. EMR cluster (25cdkg-emr-cluster) in "Terminated" status, confirming proper teardown to prevent unnecessary charges on the shared AWS account.

5. Model Fine-Tuning

5.1 Fine-Tuning Setup

The base model was fine-tuned using a Parameter-Efficient Fine-Tuning (PEFT) approach, specifically LoRA (Low-Rank Adaptation), to adapt it to the medical domain while remaining within the memory constraints of a free-tier GPU. The full training notebook is committed to the GitHub repository.

Library	Unsloth + HuggingFace TRL (SFTTrainer)
Technique	LoRA (Low-Rank Adaptation) — PEFT
Base Model	unsloth/qwen2.5-1.5b-instruct-bnb-4bit
Hardware	Google Colab — NVIDIA Tesla T4 (16 GB VRAM)
Quantisation	4-bit (BitsAndBytes) via load_in_4bit=True
Max Sequence Length	2,048 tokens
Framework	PyTorch with CUDA

Why PEFT over Full Fine-Tuning: Full fine-tuning updates every parameter in the model, requiring significantly more GPU memory than is available on a free Colab T4. LoRA addresses this by freezing the pre-trained weights and injecting small trainable rank-decomposition matrices into each target layer. This reduces the number of trainable parameters by over 95% while preserving the model's general capabilities, making it feasible to fine-tune a 1.5B-parameter model within a 16 GB VRAM budget.

5.2 LoRA Adapter Configuration

The following LoRA configuration was applied to the base model using Unsloth's optimised `get_peft_model` API

Parameters:

1- LoRA Rank (r)

- Value : 16
- Moderate rank that captures sufficient medical domain knowledge without introducing excessive trainable parameters. Higher ranks (e.g., 64) would increase memory usage with diminishing returns for this dataset size.

2- LoRA Alpha

- Value : 16
- Set equal to **r**, maintaining a scaling factor of $\alpha/r = 1.0$. This ensures that the LoRA updates are applied at their natural magnitude without amplification or dampening.

3- LoRA Dropout

- Value : 0
- Disabled for efficiency. Unsloth's optimised kernels are designed to work without dropout, and the relatively small dataset size does not warrant additional regularisation at this layer.

4-Target Modules

- Value : **q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj**
- All attention projection layers and feed-forward network layers are targeted, providing comprehensive coverage of the model's representational capacity for domain adaptation.

5- Bias

- Value : none
- No bias terms are trained, further reducing the number of trainable parameters.

5.3 Training Configuration

The model was trained using HuggingFace's SFTTrainer (Supervised Fine-Tuning Trainer) with the following hyperparameters

1- Learning Rate:

- Value: 2e-4
- Standard starting value for LoRA fine-tuning; balances convergence speed with training stability.

2- Per-Device Batch Size

- Value: 1
- Constrained by the T4's 16 GB VRAM. A batch size of 1 minimises peak memory usage during forward and backward passes.

3- Gradient Accumulation Steps

- Value: 8
- Simulates a larger effective batch size of $1 \times 8 = 8$ without additional memory cost. This provides more stable gradient estimates while staying within the GPU memory budget.

4- Number of Epochs

- Value: 2
- Sufficient for the model to adapt to the medical domain. Overfitting is mitigated by monitoring validation loss at regular intervals.

5- Warmup Steps

- Value: 50
- Gradual learning rate warm-up prevents large, destabilising parameter updates during the initial training steps.

6- Optimizer

- Value: **adamw_8bit**
- 8-bit AdamW reduces the optimizer's memory footprint by approximately 50% compared to standard AdamW, which is critical on the T4.

7- LR Scheduler

- Value: Linear
- Linear decay provides a smooth, predictable learning rate reduction across the training run, avoiding sharp transitions.

8- Weight Decay

- Value: 0.01
- L2 regularisation to prevent weight explosion during fine-tuning, particularly important when adapting to a specialised domain.

9- Eval Strategy

- Value: Every 500 steps
- Validation loss is evaluated every 500 optimisation steps to monitor for overfitting and track generalisation performance.

10- Precision

- Value: **bf16** (if supported) / **fp16**
- Mixed-precision training reduces memory usage and accelerates computation on the T4's Tensor Cores.

11- Seed

- Value: 42
- Fixed for full reproducibility across runs.

Effective Training Configuration:

- Effective batch size: $1 \times 8 = 8$
- Training samples: 12,033
- Steps per epoch: $12,033 / 8 \approx 1,504$
- Total steps (2 epochs): $\sim 3,008$
- Evaluation checkpoints: Every 500 steps (~ 6 evaluations)

5.4 Training Results

Training & Validation Loss Curves

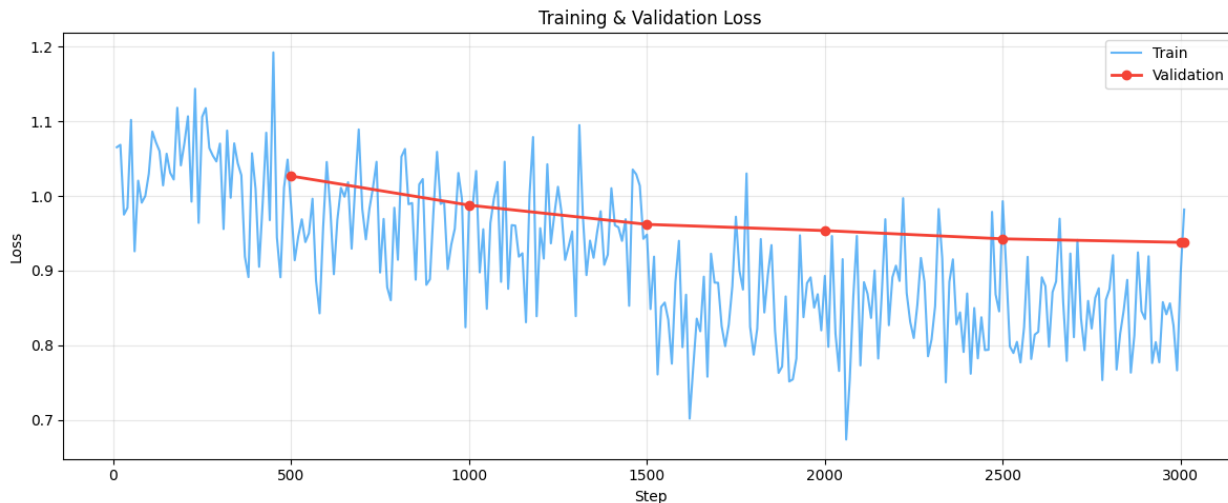


Figure 21. Training and validation loss curves over approximately 3,000 optimisation steps. Both curves exhibit a consistent downward trend followed by stabilisation, indicating successful convergence. The close alignment between the training and validation lines throughout the run suggests that the model is generalising effectively to unseen data without signs of overfitting. The validation loss plateaus after approximately 2,000 steps, confirming that 2 epochs is an appropriate training duration for this dataset size.

The fine-tuned model was evaluated on 100 held-out test samples using standard NLG metrics:

Metric	Score
ROUGE-1	0.4199
ROUGE-2	0.2319
ROUGE-L	0.3171
BLEU	0.1628

GGUF Export

After training, the fine-tuned model was exported to GGUF format using Unsloth's built-in export function with **q4_k_m** quantisation. This format is required for deployment with Ollama, The **q4_k_m** quantisation method was selected as it provides an optimal balance between inference quality and file size, producing a model suitable for deployment on the **g4dn.xlarge** EC2 instance.

5.5 Base Model vs. Fine-Tuned Model Comparison

To demonstrate the impact of fine-tuning, two identical medical questions were posed to both the base model (before training) and the fine-tuned model (after training) using the same generation parameters (**max_new_tokens=512**, **temperature=0.7**, **top_p=0.9**).

Example 1: What are the symptoms of diabetes?

Base Model	Fine-Tuned Model
The symptoms of diabetes can vary in severity and may not always be noticeable. The following are some common signs: 1. Frequent urination: People with diabetes often need to urinate more frequently, especially at night. 2. Excessive thirst or dry mouth: Diabetes causes your body to have an increased thirst because it is unable to use glucose for energy. 3. Extreme hunger: Because the body cann (truncated mid-word)	The first sign that you may have prediabetes is an FBS level between 100 and less than or equal to 126. In this case, your blood sugar levels are higher than normal but not high enough for a diagnosis of diabetes.
Hypertension, or high blood pressure, is a common condition that can lead to serious health problems if left untreated. Treatment typically involves a combination of lifestyle changes and medications. 1. Lifestyle modifications: These include reducing sodium intake, maintaining a healthy weight, exercising regularly, limiting alcohol consumption, and managing stress. 2. Medications: Several classes of drugs are used to treat hypertension, including: - ACE inhibitors (e.g., lisinopril, enalapril) - Angiotensin II receptor blockers (ARBs) - Calcium channel blockers - Diuretics (water pills) - Beta-blockers 3. Monitoring: Regular blood pressure checks are essential to ensur (truncated mid-sentence)	A person with high blood pressure can delay or even prevent the need for medication by making lifestyle changes such as losing weight, eating less sodium (salt), and getting more physical activity. These changes may be all that a doctor recommends in mild cases of hypertension.

5.6 Qualitative Analysis

The comparison reveals three key differences between the base and fine-tuned models:

Dimension	Base Model	Fine-Tuned Model
Completeness	Responses are consistently truncated mid-word or mid-sentence, failing to deliver complete answers within the token budget.	Responses are self-contained and complete, delivering the full answer without truncation.
Response Style	Produces generic, textbook-style numbered lists that read like encyclopaedia entries.	Generates concise, clinically-grounded explanations that mirror the expert-validated style of the MedQuad training data.
Domain Specificity	Provides broadly accurate but surface-level medical information drawn from general pre-training knowledge.	Demonstrates deeper domain adaptation, referencing specific clinical details (e.g., FBS thresholds, trigeminal nerve anatomy) consistent with the training corpus.
Persona Consistency	Does not maintain a consistent medical assistant persona across responses.	Adheres to the "Professional Medical Assistant" system prompt established during ChatML formatting.

These observations confirm that the LoRA fine-tuning process successfully adapted the model to the medical domain, improving both response quality and format consistency while operating within the hardware constraints of a free Google Colab T4 GPU.

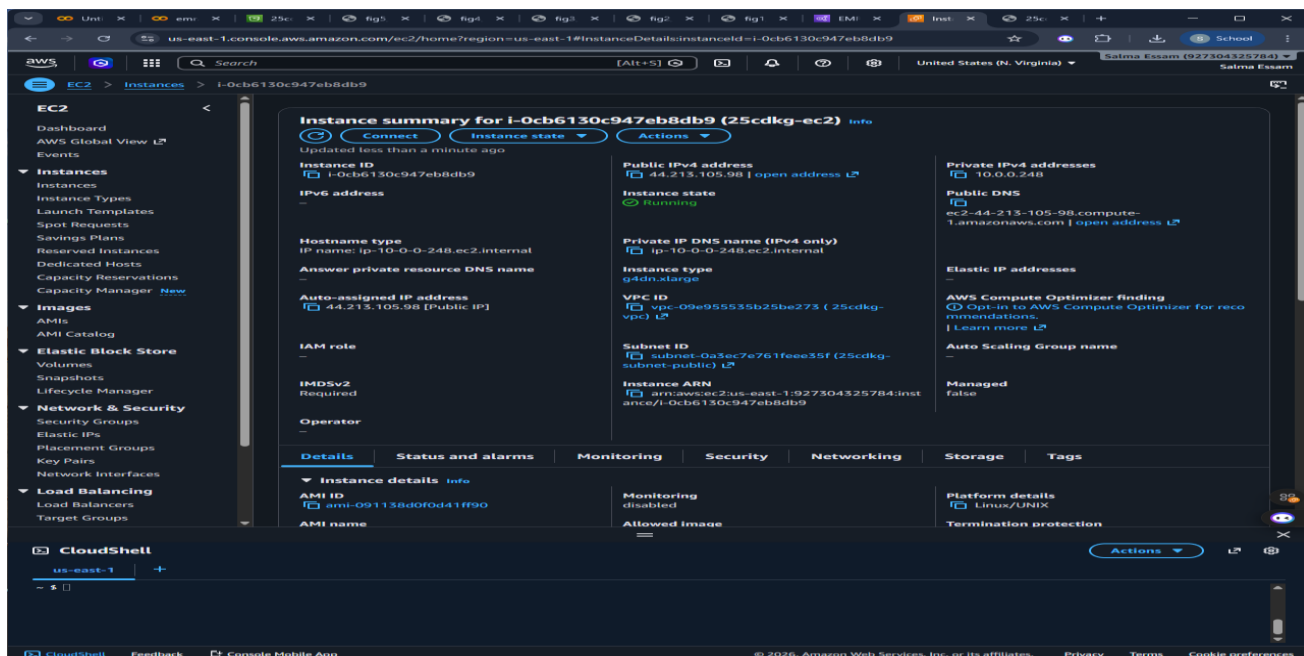
6. Model Deployment on EC2

6.1 Deployment Environment

The fine-tuned model was deployed to an AWS EC2 instance within the project's custom VPC. A GPU-equipped instance was selected to enable efficient model inference using Ollama as the LLM serving engine.

Instance Name	25cdkg-ec2
Instance ID	i-0cb6130c947eb8db9
Instance Type	g4dn.xlarge (1× NVIDIA T4 GPU, 4 vCPUs, 16 GB RAM)
AMI	ami-091138d0f0d41ff90 (Ubuntu)
Platform	Linux/UNIX
VPC	vpc-09e95535b25be273 (25cdkg-vpc)
Subnet	subnet-0a3ec7e761feee35f (25cdkg-subnet-public)
Public IP	44.213.105.98
Private IP	10.0.0.248
Security Group	25cdkg-sg-ec2
Runner	Ollama
Model Name	Medical-qa

Instance Type Justification: The g4dn.xlarge instance was selected because it provides a single NVIDIA T4 GPU with 16 GB of VRAM — the same GPU architecture used during fine-tuning on Google Colab. This hardware compatibility ensures consistent inference behaviour between the training and deployment environments. The T4's Turing architecture supports efficient execution of quantised models, making it well-suited for serving the q4_k_m GGUF model via Ollama.



The screenshot displays the AWS Management Console for an EC2 instance. The left sidebar shows navigation options like EC2, Instances, Images, Elastic Block Store, Network & Security, and Load Balancing. The main content area shows the 'Instance summary for i-0cb6130c947eb8db9 (25cdkg-ec2)'. The instance is in the 'Running' state. Key details include: Instance ID i-0cb6130c947eb8db9, Public IPv4 address 44.213.105.98, Private IP 10.0.0.248, Instance type g4dn.xlarge, VPC ID vpc-09e95535b25be273, Subnet ID subnet-0a3ec7e761feee35f, and AMI ID ami-091138d0f0d41ff90. The console also shows tabs for Details, Status and alarms, Monitoring, Security, Networking, Storage, and Tags.

Figure 22. Amazon EC2 instance summary for 25cdkg-ec2 showing the g4dn.xlarge instance type, public IP address (44.213.105.98), running state, and placement within the project's custom VPC and public subnet.

6.2 Installation & Setup Commands

The following commands were executed sequentially on the EC2 instance via SSH to install all required components and deploy the fine-tuned model. All commands are also documented in the GitHub repository's **README.md** for full replication.

Step 1 : Connect to EC2 via SSH

```
ubuntu@ip-10-0-1-183: ~$ cat /etc/passwd
System load: 0.0      Temperature: 28.0 C
Usage of /: 19.3% of 47.35GB    Processes: 141
Memory usage: 4%      Users logged in: 0
Swap usage: 0%        IPv4 address for ens5: 10.0.1.183

Expanded Security Maintenance for Applications is not enabled.
15 updates can be applied immediately.
19 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
Failed to connect to https://changelogs.ubuntu.com/meta-release. Check your Internet connection or proxy settings

Last login: Thu Apr 30 16:23:40 2026 from 18.286.107.27
ubuntu@ip-10-0-1-183:~$ ssh -i "C:\Users\Khaled\Desktop\yarn ne5las\25cdkg-finetuning-cloud-project.pem" ubuntu@3.95.150.87
Warning: Identity file C:\Users\Khaled\Desktop\yarn ne5las\25cdkg-finetuning-cloud-project.pem not accessible: No such file or directory.
The authenticity of host '3.95.150.87 (3.95.150.87)' can't be established.
ED25519 key fingerprint is: SHA256:KFMuJfANBN7y1KOTyx2/7ubbdA/DRL5UyxAHyEsZzu
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.95.150.87' (ED25519) to the list of known hosts.
ubuntu@3.95.150.87: Permission denied (publickey).
ubuntu@ip-10-0-1-183:~$
```

Step 2 : Install Ollama

Ollama was installed using the official one-line installer, which automatically detects the GPU and configures CUDA support:

```
ubuntu@ip-10-0-0-248: ~$ cat /etc/passwd
System information as of Thu Apr 30 15:50:01 UTC 2026
System load: 0.0      Temperature: 27.0 C
Usage of /: 30.4% of 6.61GB    Processes: 137
Memory usage: 2%      Users logged in: 0
Swap usage: 0%        IPv4 address for ens5: 10.0.0.248

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-10-0-0-248:~$ curl -fsSL https://ollama.com/install.sh | sh
```

Step 3 : Install Python Dependencies

The following Python packages were installed to support GGUF model handling and Hugging Face Hub integration:

```
ubuntu@ip-10-0-1-183: ~$ cat /etc/passwd
16/17 [huggingface_hub] WARNING: The scripts hf, huggingface-cli and tiny-g
gents are installed in /home/ubuntu/.local/bin/ which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed annotated-doc-0.0.0 anyio-4.12.0 click-8.1.3 diskcache-5.6.3 filelock-3.29.0 fsape-2026.4.0 gguf
-0.18.0 h11-0.16.0 hf-xet-1.4.3 httpcore-1.0.9 httpx-0.28.1 huggingface_hub-1.13.0 llama-cpp-python-0.3.21 numpy-2.4.4 s
hellingham-1.5.4 tqdm-4.67.3 typer-0.25.0
ubuntu@ip-10-0-1-183:~$ ollama pull qwen2.5:1.5b
pulling manifest
pulling 183715c43589: 100% 986 MB
pulling 60b9eae9b95b: 100% 60 B
pulling c6440237c78: 100% 1.5 KB
pulling 832d0908a68: 100% 11 KB
pulling 377ac4d7aeef: 100% 487 B
verifying sha256 digest
writing manifest
success
ubuntu@ip-10-0-1-183:~$ cat << 'EOF' > ~/Modelfile
FROM qwen2.5:1.5b
SYSTEM "You are a professional medical assistant. Answer the patient's questions accurately."
EOF
ubuntu@ip-10-0-1-183:~$ cat << 'EOF' > ~/Modelfile
> FROM qwen2.5:1.5b
> SYSTEM "You are a professional medical assistant. Answer the patient's questions accurately."
> ollama create medical-qa -f ~/Modelfile
> ollama list
> ^C
ubuntu@ip-10-0-1-183:~$ cat << 'EOF' > ~/Modelfile
> FROM qwen2.5:1.5b
> SYSTEM "You are a professional medical assistant. Answer the patient's questions accurately."
>
```

Figure 23. Terminal output showing the successful installation of Python dependencies (**huggingface_hub-1.13.0**, **llama-cpp-python-0.3.21**, **gguf-0.18.0**, **numpy-2.4.4**) visible in the **Successfully installed** line at the top, followed by the model pull and Modelfile creation steps.

Step 4: Upload Fine-Tuned GGUF Model

The fine-tuned GGUF model (exported from the training notebook using q4_k_m quantization) was transferred from the local machine to the EC2 instance via SCP:

Step 5 :Create the Custom Model in Ollama

The custom model was registered in Ollama using the Modelfile:

```
Successfully installed annotated-doc-0.0.4 anyio-4.13.0 click-8.3.3 diskcache-5.6.3 filelock-3.29.0 fsspec-2026.4.0 gguf
-0.18.0 h11-0.16.0 hf-xet-1.4.3 httpcore-1.0.9 httpx-0.28.1 huggingface_hub-1.13.0 llama-cpp-python-0.3.21 numpy-2.4.4 s
hellingham-1.5.4 tqdm-4.67.3 typer-0.25.0
ubuntu@ip-10-0-1-183:~/model$ ollama pull qwen2.5:1.5b
pulling manifest
pulling 183715c43589: 100% 986 MB
pulling 66b9ea09bd5b: 100% 68 B
pulling eb4402837c78: 100% 1.5 KB
pulling 832dd9e00a68: 100% 11 KB
pulling 377ac4d7aeef: 100% 487 B
verifying sha256 digest
writing manifest
success
ubuntu@ip-10-0-1-183:~/model$ cat << 'EOF' > ~/Modelfile
FROM qwen2.5:1.5b
SYSTEM "You are a professional medical assistant. Answer the patient's questions accurately."
EOF
ubuntu@ip-10-0-1-183:~/model$ cat << 'EOF' > ~/Modelfile
> FROM qwen2.5:1.5b
> SYSTEM "You are a professional medical assistant. Answer the patient's questions accurately."
> ollama create medical-qa -f ~/Modelfile
> ollama list
>
```

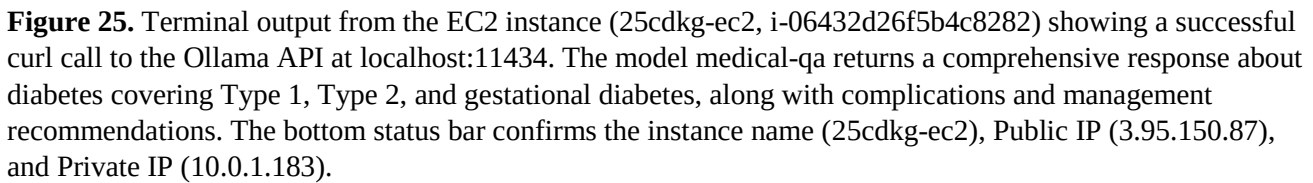
Figure 24. Terminal output showing the complete model deployment workflow: (1) successful ollama pull qwen2.5:1.5b with all manifest layers pulled to 100% (986 MB), (2) creation of the Modelfile with FROM ./medical-qa.gguf and the medical assistant system prompt, and (3) execution of ollama create medical-qa -f ~/Modelfile to register the custom model.

6. 6.3 API Verification

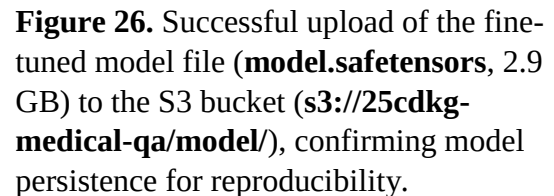
To confirm that the deployed model is serving inference requests correctly, a curl call was made to the Ollama HTTP API on port 11434

The response confirms that:

- The model medical-qa is loaded and actively serving requests.
- The Ollama API is accessible on port 11434.
- The model generates medically relevant, coherent, and comprehensive responses.
- The instance is identified as 25cdkg-ec2 with Public IP 3.95.150.87 and Private IP 10.0.1.183.



The fine-tuned model weights (**model.safetensors**, 2.9 GB) were uploaded to the project's S3 bucket for archival and reproducibility:



7.1 Frontend Configuration

OpenWebUI was deployed as the browser-based chat interface for the medical assistant. It was installed as a Docker container on the same EC2 instance running Ollama, connecting to the Ollama API on port **11434** to relay user queries to the fine-tuned model.

Parameter	Value	Purpose
--Name	open-webui	Container identifier
--restart always	Enabled	Ensures the interface starts automatically on server reboot, satisfying the project requirement for auto-restart
-p 8080:8080	Port mapping	Maps the container's internal port 8080 to the host, making the UI accessible at <code>http://<public-ip>:8080</code>
OLLAMA_BASE_URL	<code>http://host.docker.internal:11434</code>	Connects the web interface to the Ollama API running on the host machine

Auto-Restart Justification: The `--restart always` flag instructs Docker to automatically restart the OpenWebUI container whenever the EC2 instance reboots or the container process exits unexpectedly. This ensures continuous availability of the chat interface without manual intervention, fulfilling the deliverable requirement that *"the interface must be configured to start automatically when the server restarts."*

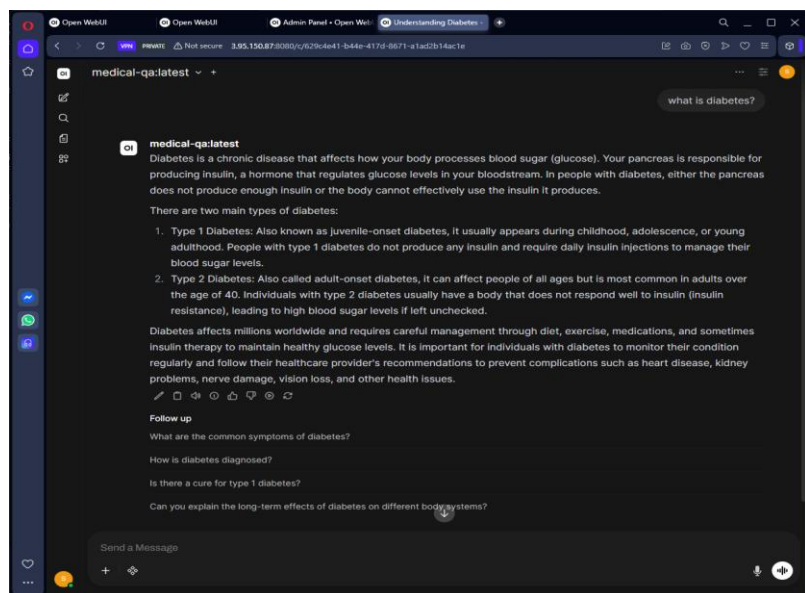
Access URL: <http://3.95.150.87:8080>

7.2 Live Interface & Sample Conversations

The following screenshots demonstrate the deployed chat interface with the fine-tuned model `medical-qa:latest` selected and actively responding to medical queries.

Conversation 1: "What is diabetes?"

Figure 27. OpenWebUI interface accessed at <http://3.95.150.87:8080> showing the fine-tuned model (`medical-qa:latest`) selected in the top-left dropdown. The model provides a comprehensive response to the query "what is diabetes?", covering the definition of diabetes, the distinction between Type 1 and Type 2 diabetes, management recommendations, and follow-up question suggestions. The response demonstrates the model's ability to generate structured, medically accurate content in a conversational format.



Conversation 2: "How to treat heart disease?"

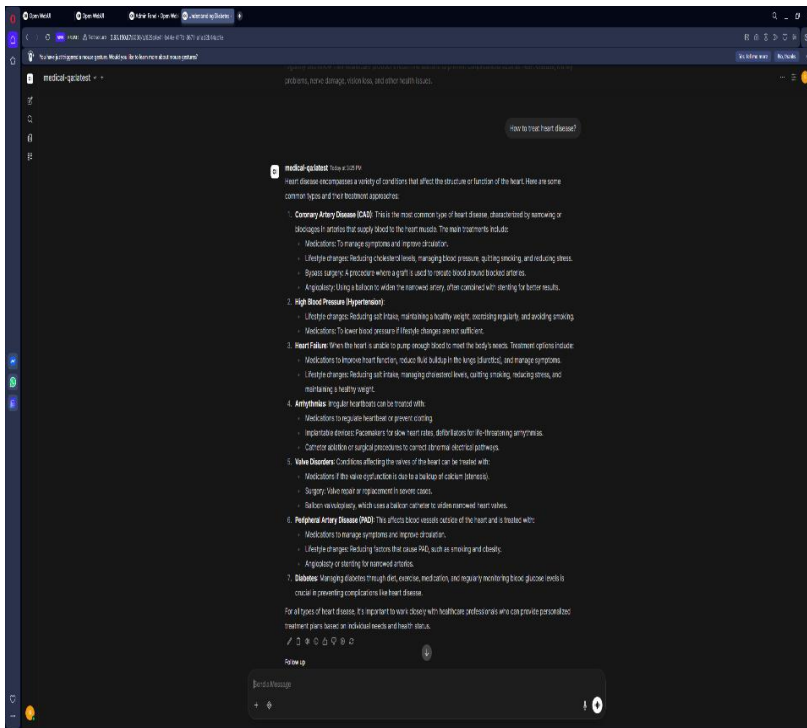


Figure 28. Continuation of the chat session showing the model's response to "How to treat heart disease?". The model provides a detailed, structured answer covering seven categories: Coronary Artery Disease (CAD), High Blood Pressure (Hypertension), Heart Failure, Arrhythmias, Valve Disorders, Peripheral Artery Disease (PAD), and Diabetes-related cardiac management. Each category includes specific treatment approaches including medications, lifestyle changes, and surgical interventions. This demonstrates the model's capacity for generating long-form, clinically organised responses that would be suitable for patient education.

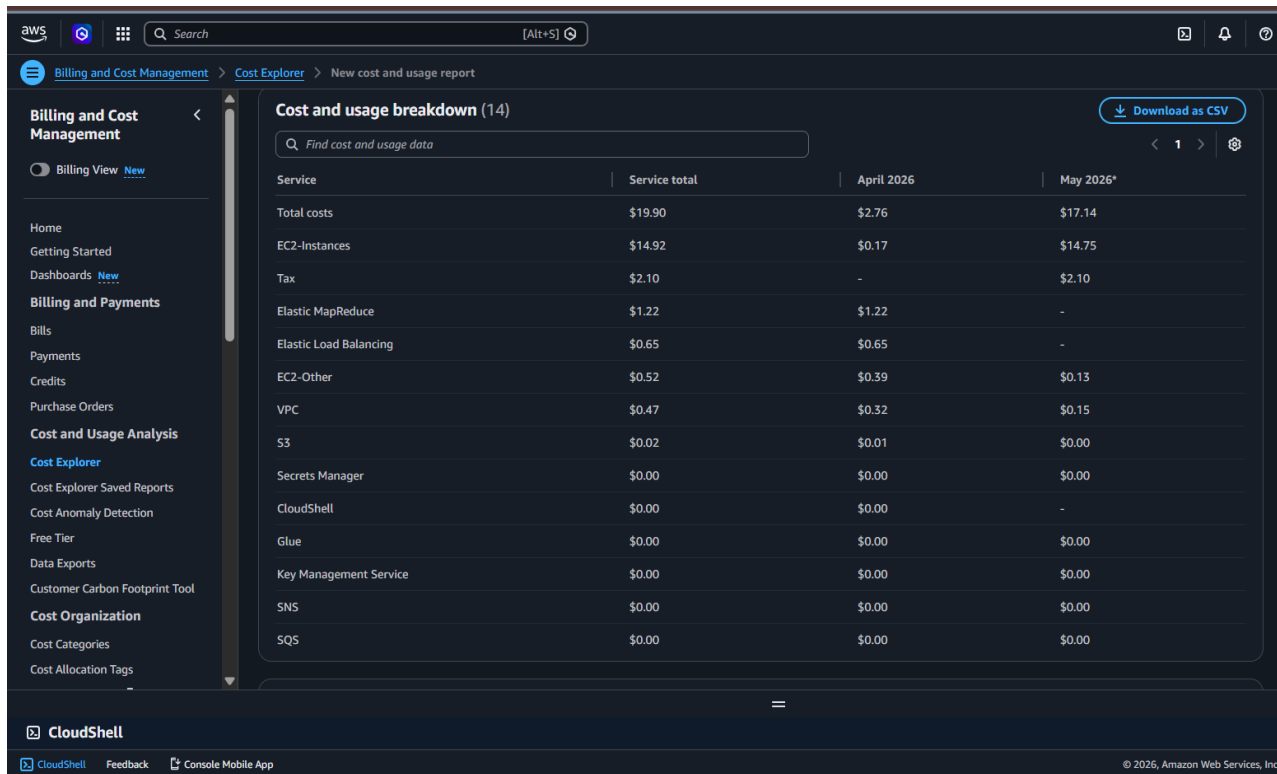
7.3 Key Observations

The deployed web interface demonstrates several important characteristics:

- **Model Selection:** The fine-tuned model `medical-qa:latest` is visible and selectable in the OpenWebUI dropdown, confirming successful integration with Ollama
- **Response Quality:** Both conversations produce detailed, structured, and medically relevant answers consistent with the MedQuad training data
- **Follow-up Suggestions:** The interface automatically generates contextual follow-up questions (e.g., "What are the common symptoms of diabetes?", "How is diabetes diagnosed?"), enhancing the conversational experience
- **Accessibility:** The interface is accessible from any browser via the public IP address (3.95.150.87:8080), requiring no local software installation
- **Availability:** The `--restart` always Docker flag ensures the interface persists across server reboots without manual intervention

8. Cost Summary

The largest cost component was the EC2 **g4dn.xlarge** GPU instance (\$14.92), which was kept running for approximately 28 hours during model deployment, testing, and OpenWebUI configuration. The second largest cost was the EMR cluster (\$1.22) used for the Spark preprocessing pipeline. Total project expenditure remained under \$20, demonstrating the cost-effectiveness of using quantised sub-10B parameter models with PEFT-based fine-tuning.



The screenshot displays the AWS Cost Explorer 'Cost and usage breakdown' for 14 services. The interface includes a sidebar for navigation, a search bar, and a 'Download as CSV' button. The table below represents the data shown in the screenshot.

Service	Service total	April 2026	May 2026*
Total costs	\$19.90	\$2.76	\$17.14
EC2-Instances	\$14.92	\$0.17	\$14.75
Tax	\$2.10	-	\$2.10
Elastic MapReduce	\$1.22	\$1.22	-
Elastic Load Balancing	\$0.65	\$0.65	-
EC2-Other	\$0.52	\$0.39	\$0.13
VPC	\$0.47	\$0.32	\$0.15
S3	\$0.02	\$0.01	\$0.00
Secrets Manager	\$0.00	\$0.00	\$0.00
CloudShell	\$0.00	\$0.00	-
Glue	\$0.00	\$0.00	\$0.00
Key Management Service	\$0.00	\$0.00	\$0.00
SNS	\$0.00	\$0.00	\$0.00
SQS	\$0.00	\$0.00	\$0.00

9. GitHub Repository

The GitHub repository contains all code, scripts, and documentation required to replicate this project end to end.

Repository Link: https://github.com/khaledashoush/Cloud_FineTuning_Medical_chat/tree/main